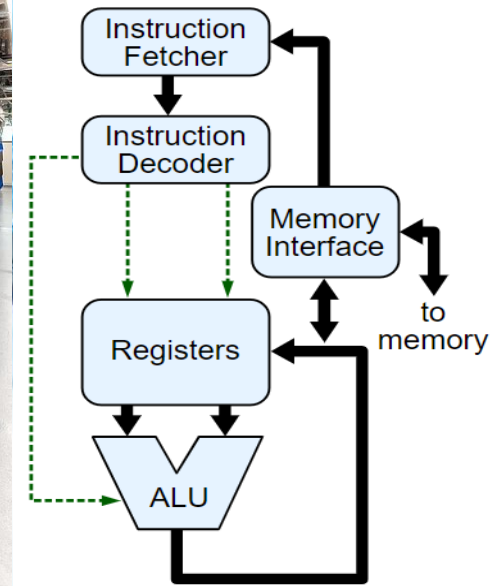
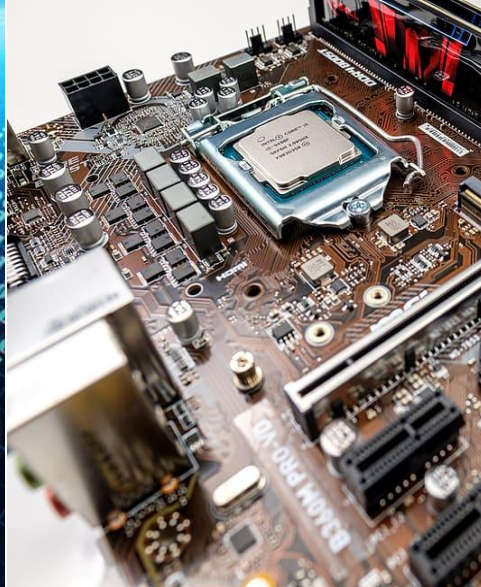




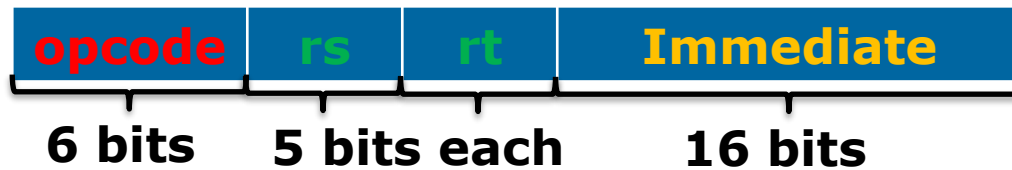
Recap

- ▶ The MIPS32 CPU has 32 registers: \$0, ..., \$31, each of size 32 bits.
- ▶ The CPU has three instruction types:
 - **R**-ithmetic (between register) operations.
 - add \$8, \$1, \$2
 - sub \$12, \$6, \$3
 - **I**-mmediate (including fixed numbers) operations.
 - addi \$15, \$15, -1
 - beq \$10, \$14, 0x00000002
 - **J**-ump operations.
 - j 0x00400050
 - jal 0x0040007c

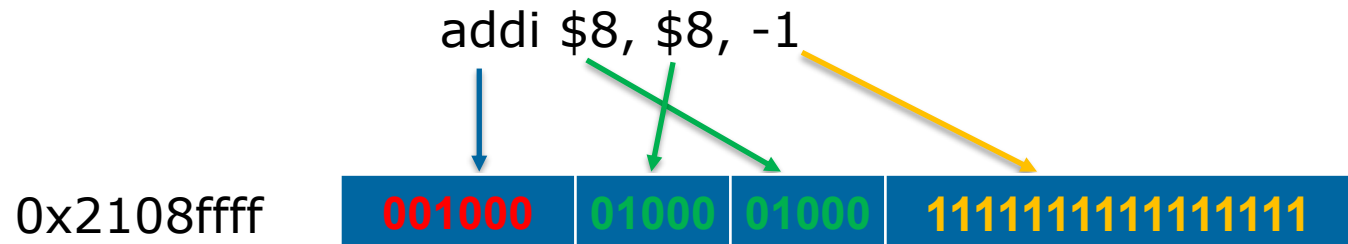


I-immediate instructions

- Work with a 16-bit *immediate* value



$R[rt] = \text{func}(\text{Imm}, R[Rs])$ *



Add Immediate `addi` I $R[rt] = R[rs] + \text{SignExtImm}$ (1,2) 8_{hex}

Memory addressing

- ▶ Memory addresses are 32-bit long.
 - And so are the registers.
- ▶ A single way to compute a memory address:

Base (register value) + Displacement (Imm value).



0x20(\$8)

- ▶ If $\$8 = 0x10010000$, then $0x20(\$8)$ is the address $0x10010020$.

I-immediate memory instructions

- Working with integer arrays:
 - $A[8] = y$;
 - Say \$8 stores the address of A and \$9 stores y

Store Word

sw

I $M[R[rs] + \text{SignExtImm}] = R[rt]$ (2) $2b_{\text{hex}}$

sw \$9, 0x20(\$8)

Must be
word-aligned

0xad090020



$-z = A[10];$ # store A[10] into \$10.

Load Word

lw

I $R[rt] = M[R[rs] + \text{SignExtImm}]$ (2) 23_{hex}

lw \$10, 0x28(\$8)

0x8d0a0028

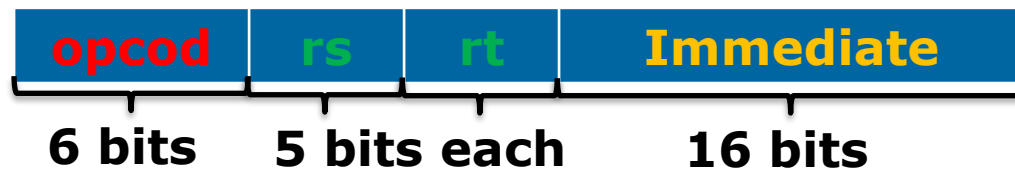


I-mmediate memory instructions

- ▶ Also works with byte and half word granularities
 - Load/store byte (lb/sb).
 - Load/store half-word (lh/sh). 
 - Are sign-extended.
- ▶ Support for zero-extensions (lbu/lhu/sbu/shu).

I-immediate value assignment

- ▶ In Java, we may want to do
 - `int x = 7654321;`
- ▶ How would that be translated to MIPS?



CPU instructions →

Basic	Source
<code>lui \$1, 0x00000074</code>	<code>6: li \$8, 7654321</code>
<code>ori \$8, \$1, 0x0000cbb1</code>	

← Pseudo instruction

$7654321_{10} = 0x74cbb1$

- ▶ Load upper immediate (lui) changes the most significant 16 bits and resets the lower 16.
 $\$1 = 0x740000$
- ▶ Or immediate (ori) performs bitwise-or with the immediate 0xcbb1.

Adding *unsigned* immediate?

- ▶ What is the value of \$8 after the following?

addiu \$0, \$0, 5

addiu \$8, \$0, 5

addiu \$8, \$8, 0xFFFF

Add Imm. Unsigned `addiu` I $R[rt] = R[rs] + \text{SignExtImm}$ (2) 9_{hex}

(2) $\text{SignExtImm} = \{ 16\{\text{immediate}[15]\}, \text{immediate} \}$

Lecture 3: Immediate MIPS Instructions



Adding *unsigned* immediate?

- What is the value of \$8 after the following?

`addiu $0, $0, 5`

`addiu $8, $0, 5`

`addiu $8, $8, 0xFFFF`

The correct answer
is 4

Add Imm. Unsigned `addiu` I $R[rt] = R[rs] + \text{SignExtImm}$ (2) 9_{hex}

(2) $\text{SignExtImm} = \{ 16\{\text{immediate}[15]\}, \text{immediate} \}$

Despite its name, add immediate unsigned (`addiu`) is used to add constants to signed integers when we don't care about overflow. MIPS has no subtract immediate instruction, and negative numbers need sign extension, so the MIPS architects decided to sign-extend the immediate field.

[Computer Organization and Design: The Hardware/software Interface By David A. Patterson, John L. Hennessy](#)

Why did I accept 65540 as correct?

- Let's take a look into the MARS console:

Basic	Source
<code>addiu \$0,\$0,0x00000005</code>	5: <code>addiu \$0, \$0, 5</code>
<code>addiu \$8,\$0,0x00000005</code>	6: <code>addiu \$8, \$0, 5</code>
<code>lui \$1,0x00000000</code>	7: <code>addiu \$8, \$8, 0xFFFF</code>
<code>ori \$1,\$1,0x0000ffff</code>	
<code>addu \$8,\$8,\$1</code>	

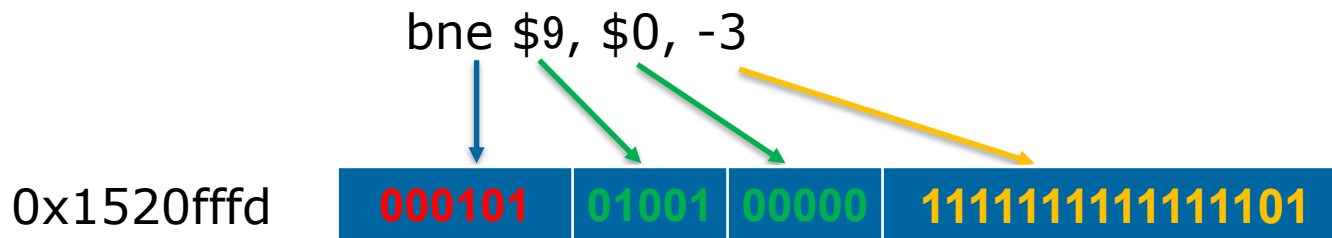
- The *assembler* replaced `addiu $8, $8, 0xFFFF` with three instructions that together add $0xFFFF = 65535_{10}$ to \$8!
- On the other hand:

Code	Basic	Source
<code>0x2508ffff</code>	<code>addiu \$8,\$8,0xffffffff</code>	2: <code>addiu \$8, \$8, -1</code>

- MARS replaced `addiu $8, $8, -1` with `addiu $8, $8, 0xffff!`

I-immediate branching instructions

- ▶ Essential for implementing loops



$$\text{Branch On Not Equal}_{\text{bne}} \quad \text{I} \quad \begin{array}{l} \text{if}(R[\text{rs}] \neq R[\text{rt}]) \\ \text{PC} = \text{PC} + 4 + \text{BranchAddr} \end{array} \quad (4) \quad 5_{\text{hex}}$$

(4) BranchAddr = { 14{immediate[15]}, immediate, 2'b0 }



I-immediate branching instructions

► More branch instructions:

Branch On Equal `beq` I if($R[rs] == R[rt]$)
PC = PC + 4 + BranchAddr (4) 4_{hex}

- Branch on comparison with zero (`bgez`, `bgtz`, `blez`, `bltz`).
- Trivia: `bgez` and `bltz` both have opcode=1. How can this be?

► How to translate if ($x > y$)?

- Pseudo instruction: branch greater than (`bgt`)
- `bge`, `ble`, `blt`

0x016a082a	<code>slt \$1,\$11,\$10</code>	13: <code>bgt \$10, \$11, loop</code>
0x1420ffff	<code>bne \$1,\$0,0xffffffff</code>	

Keep in mind when programming

```
for (int i=0; i<10; ++i){  
    func();  
}
```

Basic	Source
addiu \$13,\$0,0x00000000	7: li \$13, 0
addiu \$14,\$0,0x0000000a	8: li \$14, 10
jal 0x00400024	10: jal func
addi \$13,\$13,0x00000001	11: addi, \$13, \$13, 1
slt \$1,\$13,\$14	12: blt \$13, \$14, loop
bne \$1,\$0,0xffffffffc	

```
for (int i=10; i>0; --i){  
    func();  
}
```

Basic	Source
addiu \$13,\$0,0x0000000a	8: li \$13, 10
jal 0x00400024	10: jal func
addi \$13,\$13,0xffffffff	11: addi, \$13, \$13, -1
bgtz \$13,0xffffffffd	12: bgtz \$13, loop

A few announcements

- ▶ I will be publishing the solution to CW1 soon.
 - I still highly recommend that you try it yourself before peeking. It's short and well worth it if you want to nail the exam!
- ▶ Soon thereafter, CW2 will come out.
 - Mips programming, pseudo instructions, and other stuff.
 - Longer than CW1, and can be quite challenging!
- ▶ The Pulse survey is available:

